SUMMER TRAINING/INTERNSHIP REPORT

(Term Aug-Dec 2026)

ON

DATA STRUCTURES AND ALGORITHMS

Project: HOSTEL BOOKING MANAGEMENT SYSTEM

Submitted by

Name: SHRISTI SINGH

Registration Number: 12419008

School of Computer Science and Engineering
Lovely Professional University, Punjab

DECLARATION

I hereby declare that the **Summer Training/Internship Report** entitled **"Data Structures and Algorithms"** and the project titled **"Hostel Booking Management System"** have been prepared and submitted by me as part of the academic requirements of the **Bachelor of Technology in Computer Science and Engineering at Lovely Professional University, Punjab**.

The work presented in this report is based on the learning and practical work completed by me during my summer training at **Techvanto Academy**. The training provided me with an opportunity to study and apply various concepts of Data Structures and Algorithms through practical exercises and projects.

I further declare that this report represents my own academic work and has been prepared for submission to **Lovely Professional University**. The information and work presented in this report are based on my learning and project development during the training period.

I take full responsibility for the authenticity and accuracy of the work presented in this report.

Name: Shristi Singh
Registration Number: 12419008
Programme: B.Tech. Computer Science and Engineering
University: Lovely Professional University, Punjab
Date: July 20, 2026

CERTIFICATE

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Lovely Professional University** and **The School of Computer Science and Engineering** for providing me with the opportunity to undertake this Summer Training/Internship as a part of my academic curriculum.

I am thankful to **Techvanto Academy** for providing me with the opportunity to learn **Data Structures and Algorithms** through practical sessions and projects. The training helped me strengthen my programming, problem-solving, and practical DSA skills.

I would also like to thank everyone who supported and guided me during the completion of my training, project, and this report.

Shristi Singh
Registration Number: 12419008

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION OF ORGANIZATION

1. LOVELY PROFESSIONAL UNIVERSITY

Lovely Professional University (LPU), Punjab is the academic institution where I am pursuing my **Bachelor of Technology in Computer Science and Engineering**. The university provides an academic environment that focuses on developing students' technical knowledge, practical skills, and professional capabilities.

As a part of the academic curriculum, students are provided opportunities to gain practical exposure through training and other learning activities. The Summer Training/Internship is one such opportunity that helps students enhance their technical understanding beyond regular classroom learning.

The training undertaken during the summer period contributes to the overall academic and professional development of the student by providing exposure to practical learning.

---

2. SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

The **School of Computer Science and Engineering** at Lovely Professional University is the academic school under which the Computer Science and Engineering programme is offered.

The school provides students with opportunities to develop knowledge in different areas of computer science and encourages practical learning along with academic study. Through academic courses, practical activities, projects, and training opportunities, students can develop technical and problem-solving skills relevant to the field of computer science.

The Summer Training/Internship undertaken as part of the academic curriculum provides an opportunity to gain additional practical exposure in a technical area.

---

3. TECHVANTO ACADEMY

Techvanto Academy was the training organization where I completed my Summer Training/Internship. The training was focused on **Data Structures and Algorithms**, providing an opportunity to learn and practice fundamental concepts through a practical approach.

The training environment focused on understanding programming concepts, Data Structures, Algorithms, and their practical implementation. The training helped in developing a better understanding of how theoretical concepts can be applied while solving programming problems.

4. ROLE OF THE TRAINING ORGANIZATION

Techvanto Academy played the role of the **training organization** for the Summer Training/Internship. The training provided additional practical exposure alongside the academic learning received at Lovely Professional University.

The training helped bridge the gap between theoretical understanding and practical implementation by providing structured learning and programming practice. It also provided an opportunity to improve technical understanding and problem-solving skills in the area of Data Structures and Algorithms.

CHAPTER 2: SUMMER TRAINING COURSE/INTERNSHIP CONTENT DETAIL

1. INTRODUCTION TO DATA STRUCTURES

Data Structure is a way of organizing and storing data so that it can be accessed and processed efficiently. Different types of data require different methods of organization.

During the training, I learned that selecting the appropriate Data Structure depends on the type of problem and the operations that need to be performed. The major Data Structures covered during the training included arrays, linked lists, stacks, queues, trees, and hashing. The practical understanding of these structures helped in developing better programming logic and solving problems in a systematic way.

---

2. INTRODUCTION TO ALGORITHMS

An algorithm is a step-by-step procedure used to solve a particular problem.

A good algorithm should be clear, finite, and produce the required result for a given problem. During the training, algorithms were studied along with Data Structures to understand how data can be processed efficiently. For example, searching algorithms can be used to find an element from a collection, while sorting algorithms can arrange data in a particular order. Understanding algorithms helped me develop a structured approach towards solving programming problems.

---

3. ARRAYS

An array is a linear Data Structure used to store multiple elements of the same data type. The elements of an array are stored using indexes. The first element generally has index 0.

Example:

10 20 30 40 50

0 1 2 3 4

Arrays allow direct access to elements using their index, which makes accessing a particular element fast. Arrays are useful when the number of elements is known in advance and data needs to be accessed frequently.

Advantages

- Easy to implement.
- Direct access using index.
- Simple traversal.

Limitation

The size of a basic array is fixed after it is created.

---

4. STRINGS

A string is a sequence of characters used to store text.

Strings are commonly used for storing information such as names, addresses, messages, and other textual data. During training, string operations were used in programming problems involving comparison, searching, and manipulation of text.

In Java, strings can be created using the String class.

Example:

String name = "Shristi";

Strings are especially useful in management systems because information such as student names, course names, and room-related details often needs to be stored and compared.

---

5. SEARCHING TECHNIQUES

Searching is the process of finding a particular element from a collection of data.

One basic searching technique is **Linear Search**, in which elements are checked one by one until the required element is found.

Example:

10 → 20 → 30 → 40 → 50

↑

Search

If the required element is 30, the algorithm checks the elements sequentially until 30 is found. Searching is an important operation because many applications require finding a particular record from stored data.

---

6. SORTING TECHNIQUES

Sorting is the process of arranging data in a specific order, usually ascending or descending.

Some commonly studied sorting techniques include:

- Bubble Sort
- Selection Sort
- Insertion Sort

For example:

Before:

40 10 30 20

After:

10 20 30 40

Sorting makes data easier to read, search, and process. The choice of sorting technique depends on the size of the data and the requirements of the problem.

---

7. LINKED LISTS

A linked list is a linear Data Structure made up of nodes.

Each node stores data and a reference to another node.

A simple linked list can be represented as:

[Data|Next] → [Data|Next] → [Data|Next] → NULL

Unlike arrays, linked-list elements do not need to be stored in continuous memory locations. Linked lists are useful when elements need to be dynamically added or removed.

During the training, linked lists helped in understanding the concepts of nodes, references, traversal, insertion, and deletion.

---

8. DOUBLY LINKED LISTS

A doubly linked list is an extension of a linked list where each node contains two references.

These references point towards:

- The previous node
- The next node

It can be represented as:

NULL ← Node ⇄ Node ⇄ Node → NULL

This allows the structure to be traversed in both directions.

In the Hostel Booking Management System, the Node class contains next and previous references, making the linked structure suitable for organizing hostel room information.

This provided practical understanding of how doubly linked lists can be used in an application.

---

9. STACKS

A stack is a linear Data Structure that follows the **LIFO (Last In, First Out)** principle.

The element inserted last is removed first.

The main operations of a stack are:

- **Push** – adds an element.
- **Pop** – removes the top element.
- **Peek** – views the top element.

In the Hostel Booking Management System, a Stack is used for maintaining booking history so that the latest booking activity can be accessed first.

---

10. QUEUES

A queue is a linear Data Structure that follows the **FIFO (First In, First Out)** principle.

The element that enters first is removed first.

The main operations include:

- **Enqueue** – adds an element to the rear.
- **Dequeue** – removes an element from the front.
- **Peek** – views the front element.

Queues are useful in situations where elements need to be processed in the same order in which they arrive.

In the Hostel Booking Management System, a Queue is used to manage students waiting for room allocation.

---

11. CIRCULAR QUEUE

A circular queue is a modified form of a queue in which the last position is connected back to the first position. This arrangement allows the available positions at the beginning of the queue to be reused.

Circular queues are useful when memory needs to be utilized efficiently in a fixed-size queue. Understanding circular queues helped in learning how queue operations can be improved for certain applications.

---

12. RECURSION

Recursion is a programming technique in which a function calls itself to solve a smaller version of the same problem.

A recursive function generally contains:

1. A **base condition** to stop recursion.
2. A **recursive call** that solves a smaller problem.

A common example used to understand recursion is the **Tower of Hanoi** problem. Recursion is useful for problems that can naturally be divided into smaller similar problems.

During training, recursion helped in improving logical thinking and understanding how function calls are managed.

---

13. TREES

A tree is a non-linear Data Structure used to represent hierarchical relationships.A tree consists of nodes connected through edges. The top node is called the **root**. Nodes below another node are called its children, while nodes without children are called leaf nodes.

Trees are commonly used for representing hierarchical data and are important in many areas of computer science.

14. HASHING

Hashing is a technique used to store and retrieve data using a key.

A hash function converts a key into an index or location where the corresponding data can be stored.

Hashing can provide fast data retrieval when an appropriate key is available. Hashing is useful in applications where records need to be accessed using unique identifiers such as student IDs.

---

15. TIME AND SPACE COMPLEXITY

Time complexity describes how the running time of an algorithm changes as the amount of input increases.

Space complexity describes how much additional memory an algorithm requires.

Common complexity notations include:

- **O(1)** – Constant
- **O(n)** – Linear
- **O(log n)** – Logarithmic
- **O(n²)** – Quadratic

For example, if an algorithm checks every element of an array once, its time complexity is generally **O(n)**. Understanding complexity helps in comparing different algorithms and selecting a suitable solution for a problem.

---

16. PRACTICAL APPLICATION OF DATA STRUCTURES

The main objective of the training was to understand how Data Structures can be applied to practical problems.

During the training, different management-based projects were developed, including:

- Hostel Booking Management System
- Train Management System
- Hospital Management System

These projects provided practical exposure to the use of Data Structures in different situations.

The **Hostel Booking Management System** particularly demonstrates the use of a doubly linked structure for rooms, a queue for the waiting list, a stack for booking history, and an Array List for maintaining students inside rooms.

Through these practical applications, I learned that Data Structures are not only theoretical concepts but can be used to organize information and solve real-world problems in a structured manner.

CHAPTER 3: SUMMER TRAINING/INTERNSHIP PROJECT DETAIL

3.1 PROBLEM STATEMENT

Managing hostel rooms and student accommodation manually can become difficult when multiple students require rooms. It is necessary to keep track of available rooms, room capacity, students assigned to rooms, bookings, cancellations, and students waiting for accommodation.

When all available beds are occupied, there should also be an organized way to maintain the students who are waiting for a room. Similarly, when a booking is cancelled, the newly available bed should be properly reflected in the system.

The objective of this project was to develop a simple **Hostel Booking Management System** that can organize these operations using **Java and Data Structures**.

The system provides basic functionality for managing hostel rooms, student information, room booking, cancellation, waiting-list management, searching, and booking history. The project focuses mainly on demonstrating the practical application of Data Structures to a real-world problem.

3.2 PROJECT OUTCOMES

The development of the Hostel Booking Management System provided practical understanding of how Data Structures can be applied to solve a real-world management problem.

The major outcomes of the project are:

- Developed a functional hostel room management system.
- Implemented a **doubly linked structure** for maintaining hostel rooms.
- Used a **Queue** to manage students in the waiting list.
- Used a **Stack** to maintain booking history.
- Used **Array List** to maintain students assigned to rooms.

- Implemented room booking and cancellation operations.
- Implemented room availability tracking.
- Implemented searching and traversal operations.
- Generated basic booking details after successful allocation.
- Improved understanding of Java classes, objects, and encapsulation.
- Improved programming logic and problem-solving skills.
- Gained practical experience in applying Data Structures to project development.

Overall, the project helped bridge the gap between theoretical DSA concepts and their practical implementation.

3.3 TECHNOLOGIES USED

The following technologies and tools were used for developing the project:

Technology / Tool	Purpose
Java	Programming language used for implementing the project
JDK	Used for compiling and executing Java programs
Visual Studio Code	Used as the development environment
Java Collections	Used for Queue, Stack and ArrayList
Data Structures	Used for organizing and managing project data

Java: Java was used as the programming language for implementing the project. Its support for classes, objects, methods, and collection classes made it suitable for implementing the required Data Structures.

Data Structures

The main Data Structures used in the project were:

- **Doubly Linked List** – for maintaining room nodes.
- **Queue** – for the waiting list.
- **Stack** – for booking history.
- **Array List** – for maintaining students inside rooms.

The primary purpose of using these structures was to demonstrate their practical application rather than to develop a complex commercial software system.

CHAPTER 4: SOURCE CODE AND SYSTEM SNAPSHOTS

4.1 SOURCE CODE FILES

Here are the core implementation files of the Hostel Booking Management System

1. **Node.java:** This snippet represents a node of the doubly linked structure used for managing hostel rooms. It stores a Room object along with references to the next and previous nodes.

```
public class Node {

    private Room room;
    private Node next;
    private Node previous;

    public Node(Room room) {

        this.room = room;
        this.next = null;
        this.previous = null;

    }

    public Room getRoom() {
        return room;
    }
```

Figure 1: Node class implementation in Visual Studio Code

2. **Room.java:** This snippet initializes the basic information of a hostel room, including its room number, floor, capacity, and available beds. The ArrayList is used to maintain the students assigned to the room..

```
public void displayRoom() {

    System.out.println(x: "-----------------------------------");
    System.out.println("Room Number    : " + roomNumber);
    System.out.println("Floor Number   : " + floorNumber);
    System.out.println("Total Capacity : " + totalCapacity);
    System.out.println("Available Beds : " + availableBeds);
```

```java
import java.util.ArrayList;

public class Room {

    private final int roomNumber;
    private final int floorNumber;
    private final int totalCapacity;
    private int availableBeds;
    private final ArrayList<Student> students;

    public Room(int roomNumber, int floorNumber, int totalCapacity) {

        this.roomNumber = roomNumber;
        this.floorNumber = floorNumber;
        this.totalCapacity = totalCapacity;
        this.availableBeds = totalCapacity;
        this.students = new ArrayList<>();
    }
```

Figure 2: Room class and student collection implementation

3. Room.java:

This method checks whether a room has an available bed before adding a student. After successful allocation, the student is added to the Array List and the number of available beds is decreased.

```java
public boolean addStudent(Student student) {

    if (availableBeds == 0) {
        return false;
    }

    students.add(student);
    availableBeds--;

    return true;
}
```

Figure 3: Student allocation and room capacity update

4. **WaitingQueue.java:** This snippet creates a Queue to store students who are waiting for hostel accommodation. The offer() operation adds a student to the rear of the queue, following the FIFO principle.

```
private final Queue<Student> queue;

public WaitingQueue() {
    queue = new LinkedList<>();
}

public void addStudent(Student student) {

    queue.offer(student);

    System.out.println(x: "\nStudent added to Waiting List successfully.");
}
```

Figure 4: Queue implementation for the hostel waiting list

5. **WaitingQueue.java:** This method retrieves and removes the student at the front of the waiting queue using poll(). It first checks whether the queue is empty to avoid removing an unavailable element.

```
public Student getNextStudent() {

    if (queue.isEmpty()) {
        return null;
    }

    return queue.poll();
}

public boolean isEmpty() {
    return queue.isEmpty();
}
```

Figure 5: Retrieving the next student from the waiting queue

6. **BookingHistory.java:** This snippet initializes a Stack for storing booking history. The push() operation adds each new booking record to the top of the stack, following the LIFO principle. It displays the stored booking records from the Stack. The pop() operation removes records from the top, so the most recently added history is displayed first.

```java
import java.util.Stack;

public class BookingHistory {

    private final Stack<String> history;

    public BookingHistory() {

        history = new Stack<>();

    }

    public void addHistory(String message) {

        history.push(message);

    }

    public void showHistory() {

        if (history.isEmpty()) {

            System.out.println(x: "\nNo Booking History Found.");

            return;

        }

        System.out.println(x: "\n======= BOOKING HISTORY =======");

        while (!history.isEmpty()) {

            System.out.println(history.pop());

        }

        System.out.println(x: "==============================");

    }

}
```

Figure 6: *Displaying booking history using Stack*

7. **Student.java:** This class represents a student and stores basic information such as ID, name, age, gender, course, and year. The constructor initializes these values when a Student object is created.

```java
public class Student {

    private final String studentId;
    private final String name;
    private final int age;
    private final String gender;
    private final String course;
    private final int year;

    public Student(String studentId, String name, int age, String gender, String course, int year) {

        this.studentId = studentId;
        this.name = name;
        this.age = age;
        this.gender = gender;
        this.course = course;
        this.year = year;
    }

    public String getStudentId() {
        return studentId;
    }

    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    public String getGender() {
        return gender;
    }

    public String getCourse() {
        return course;
    }

    public int getYear() {
        return year;
    }
```

Figure 7: *Student class implementation*

8. **Room.java:** This method searches for a student using the student ID and removes the student from the room when a match is found. After removal, the number of available beds is increased by one.

```java
public boolean removeStudent(String studentId) {

    for (Student student : students) {

        if (student.getStudentId().equals(studentId)) {

            students.remove(student);
            availableBeds++;

            return true;
        }
    }

    return false;
}
```

Figure 8: *Student removal and room availability update*

9. **main.java:** This part of the program initializes the hostel management system and provides a menu-driven interface for the user. It allows the user to select different hostel management operations.

```java
import java.util.Scanner;

public class Main {

    Run main | Debug main | Run | Debug
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        Hostel hostel = new Hostel();
        hostel.createHostel();

        while (true) {

            System.out.println(x: "\n========== HOSTEL BOOKING MANAGEMENT SYSTEM ==========");
            System.out.println(x: "1. Book Room");
            System.out.println(x: "2. Cancel Booking");
            System.out.println(x: "3. Search Student");
            System.out.println(x: "4. Display Hostel");
            System.out.println(x: "5. Display Statistics");
            System.out.println(x: "6. Show Booking History");
            System.out.println(x: "7. Add Complaint");
            System.out.println(x: "8. Display Complaints");
            System.out.println(x: "9. Display Waiting List");
            System.out.println(x: "10. Exit");
            System.out.print(s: "Enter your choice: ");
```

Figure 9: *Main class and menu-driven interface*

10. **Hostel.java:** This method creates the hostel rooms and stores each room inside a Node. The nodes are connected using next and previous references, implementing the doubly linked structure used for managing hostel rooms.

```
public void createHostel() {

    int roomNumber = 101;

    for (int floor = 1; floor <= 3; floor++) {

        for (int i = 1; i <= 10; i++) {

            int capacity;

            if (i <= 3) {
                capacity = 1;
            }
            else if (i <= 7) {
                capacity = 2;
            }
            else {
                capacity = 3;
            }

            Room room = new Room(roomNumber, floor, capacity);

            Node newNode = new Node(room);

            if (head == null) {

                head = newNode;

            } else {

                Node temp = head;

                while (temp.getNext() != null) {
                    temp = temp.getNext();
                }

                temp.setNext(newNode);
                newNode.setPrevious(temp);

            }

            roomNumber++;

        }
```

Figure 10: *Hostel room creation using a doubly linked structure*

4.2 SYSTEM SNAPSHOTS

The Hostel Booking Management System is a menu-driven Java console application. The user can select different operations such as booking a room, cancelling a booking, searching for a student, displaying hostel information, viewing statistics, managing complaints, and checking the waiting list. The system processes each operation using the Data Structures implemented in the project and displays the corresponding result in the console.

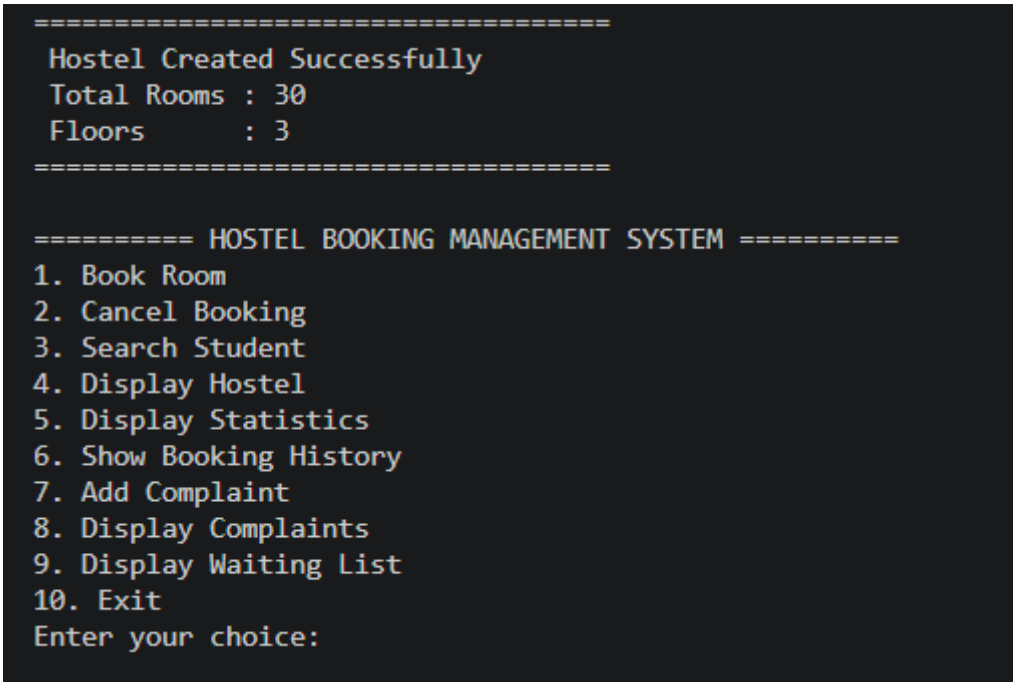

Figure 11: Hostel creation and main menu of the Hostel Booking Management System

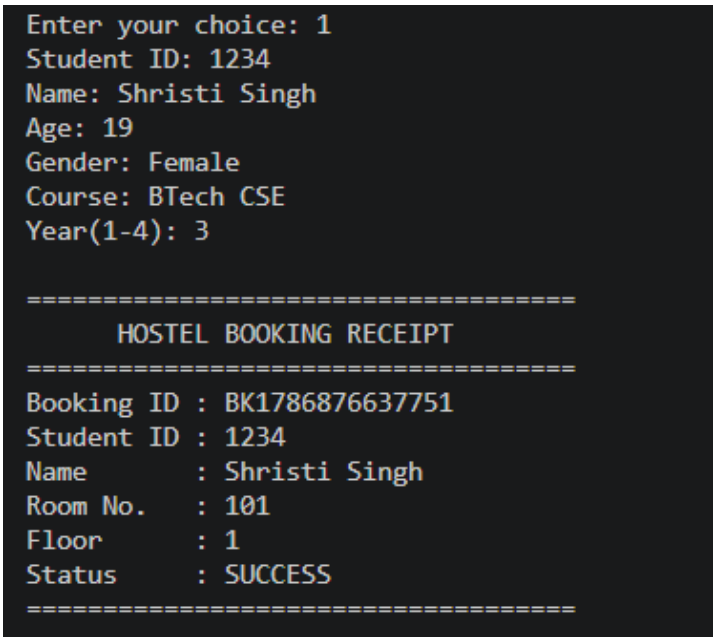

Figure 12: Successful room booking and booking receipt

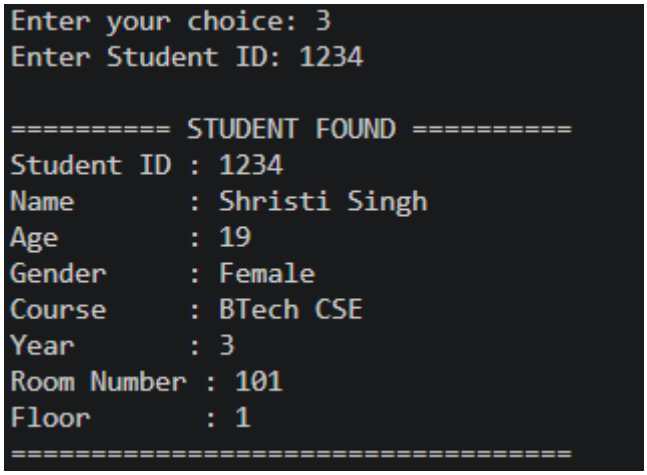

Figure 13: Search Operation performed on student

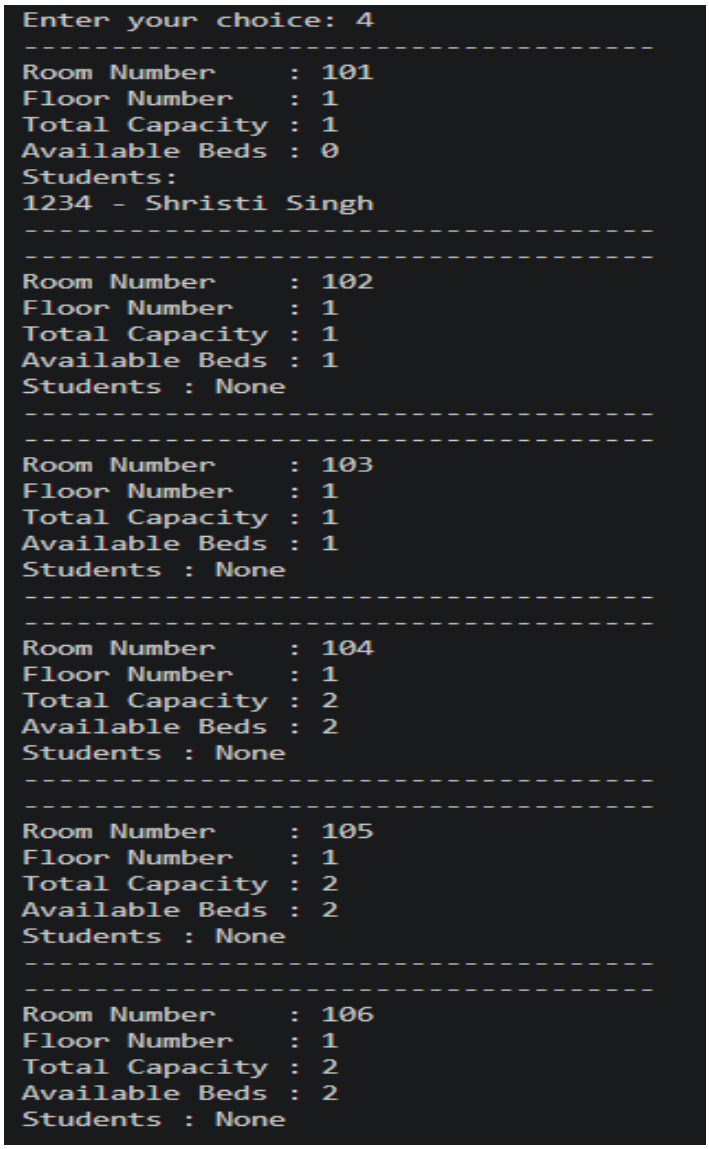

Figure 14: Hostel displayed

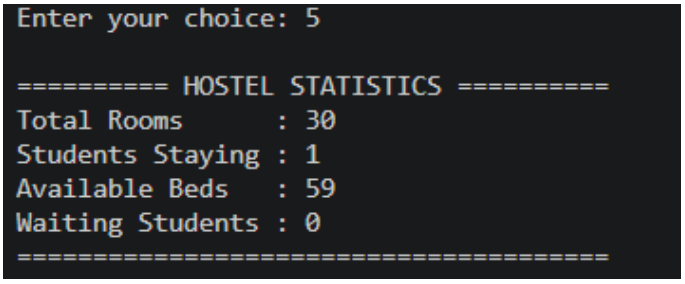

Figure 15: Hostel statistics

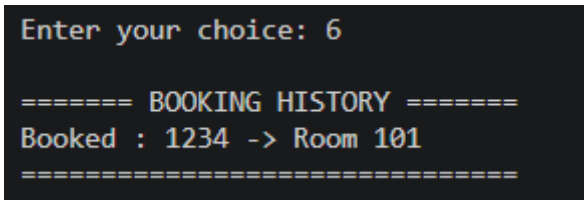

Figure 16: Booking History

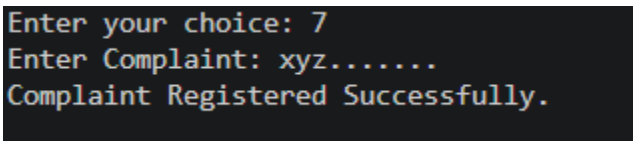

Figure 17: Complaint Management

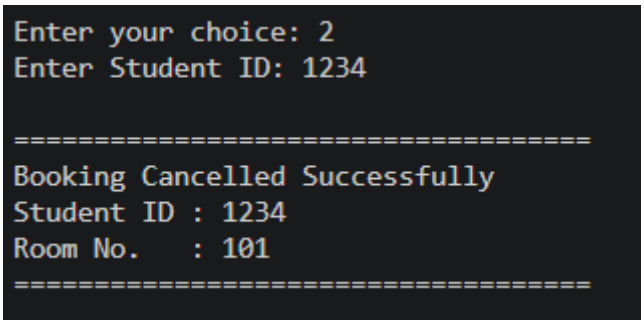

Figure 18: Cancellation Of Student Record

Figure 18: Exit (Program ended)

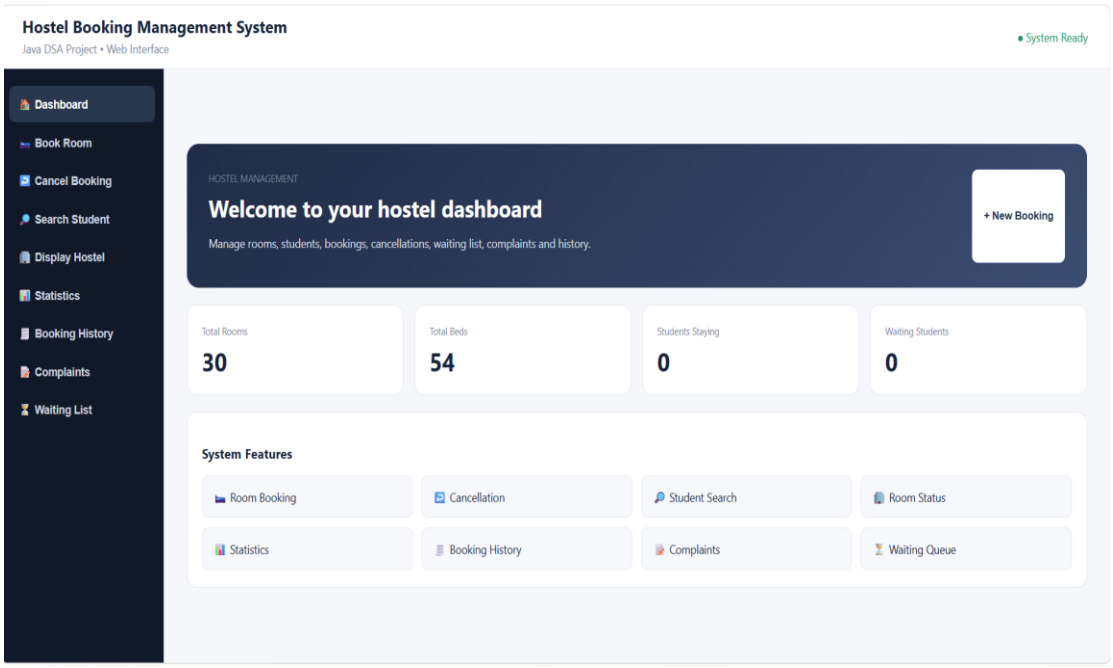

Figure 18: Optional Web-Based Interface Provided for Easier Demonstration of Project Functionalities; the Core Implementation is Java-Based and Console-Driven

GitHub Link: https://github.com/singh882/Hostel-Booking-Management-System2

CHAPTER 5: BIBLIOGRAPHY

1. **Schildt, H.** *Java: The Complete Reference*. McGraw Hill Education.
2. **Lafore, R.** *Data Structures and Algorithms in Java*. Sams Publishing.
3. **Oracle Corporation.** *Java Documentation*. Oracle Documentation.
4. **Java Platform, Standard Edition Documentation.** Java Collections Framework and Java Programming Concepts.
5. **GeeksforGeeks.** *Data Structures and Algorithms*. Available at: GeeksforGeeks.
6. **Techvanto Academy.** Training materials and practical learning resources provided during the Summer Training/Internship on Data Structures and Algorithms.
7. **Lovely Professional University.** Summer Training/Internship Report Format and academic guidelines.